

JHB Automations — System Architecture

Version: 1.0 · **Status:** Living document · **Audience:** Developers, designers, PMs, future team members

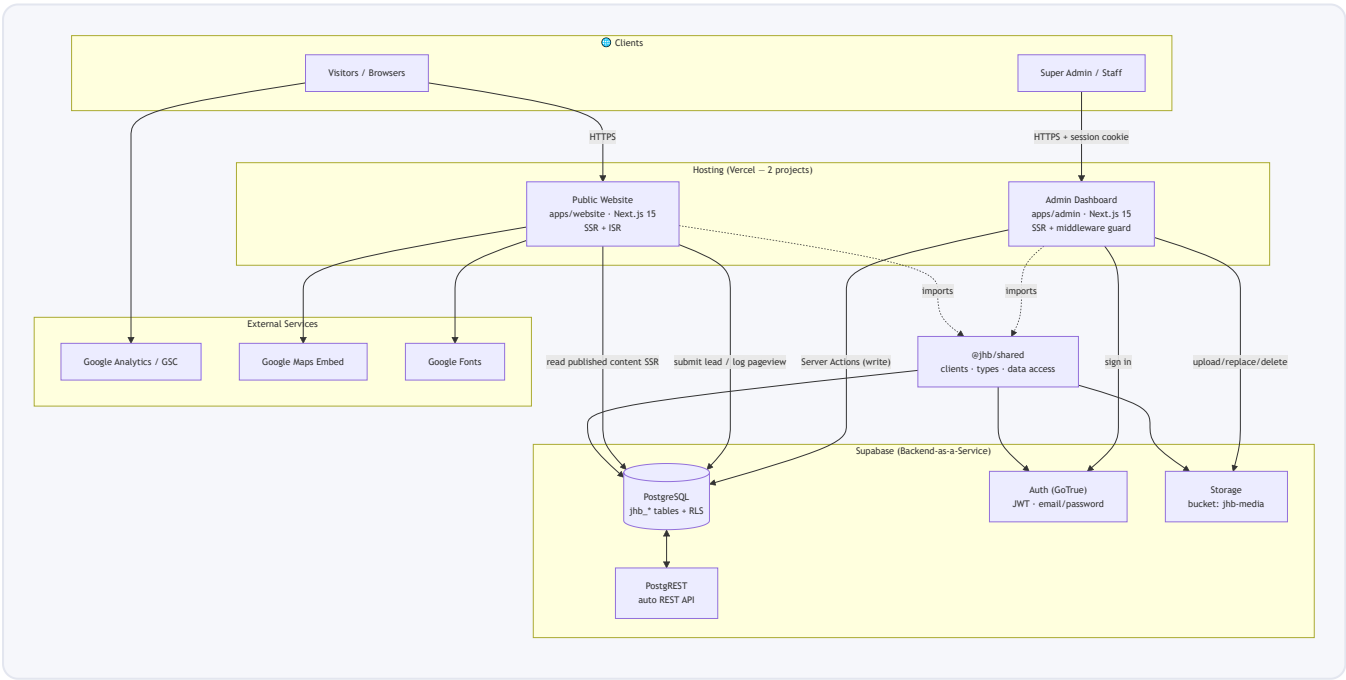
This document describes the **as-built** architecture. Where the system differs from a conventional Node/Express stack, the real design is documented first and the conventional equivalent is noted as an alternative. Items not yet built are tagged  **Planned**.

0. TL;DR — What this actually is

A **monorepo** containing **two independent Next.js 15 apps** that share one TypeScript package and **one Supabase project** as the backend (Postgres + Auth + Storage + Row-Level Security).

- **Public Website** (`apps/website` , port 3000) — marketing site, SSR, reads published content.
- **Admin Dashboard** (`apps/admin` , port 3001) — Super-Admin CMS, auth-guarded, writes content.
- **@jhb/shared** (`packages/shared`) — Supabase clients, data access, types, content models.
- **Backend = Supabase (BaaS)**: Postgres DB, GoTrue Auth (JWT), Storage bucket, PostgREST API, RLS for authorization. Mutations run through **Next.js Server Actions**; the "API layer" is Server Actions + Supabase PostgREST rather than a standalone Express/Nest service.

1. High-Level Architecture



Layer responsibilities

Layer	Implementation	Notes
Public Website	<code>apps/website</code> (Next 15 App Router, RSC, Frammer Motion)	SSR reads from Supabase via <code>@jhb/shared</code> server fns
Admin Dashboard	<code>apps/admin</code> (Next 15)	<code>middleware.ts</code> guards all routes except <code>/login</code>
Database	Supabase Postgres, <code>jhb_</code> prefixed tables	shared project; RLS on every table
Authentication	Supabase Auth (GoTrue, JWT) + <code>@supabase/ssr</code> cookie sessions	roles in <code>jhb_profiles</code>
Media Storage	Supabase Storage bucket <code>jhb-media</code> (public)	client-side compression before upload
API Layer	Next.js Server Actions (writes) + Supabase server client (reads) + PostgREST	no separate API server
SEO Layer	<code>generateMetadata</code> , <code>sitemap.ts</code> , <code>robots.ts</code> , JSON-LD, <code>jhb_seo</code> , link manager	per-page canonical + schema
Analytics Layer	<code>jhb_pageviews</code> (first-party) + admin <code>/analytics</code> ; GA/GSC 📡	<code>PageViewTracker</code> beacon
Lead Management	<code>jhb_leads</code> + public contact form + admin <code>/leads</code>	status workflow new→contacted→closed

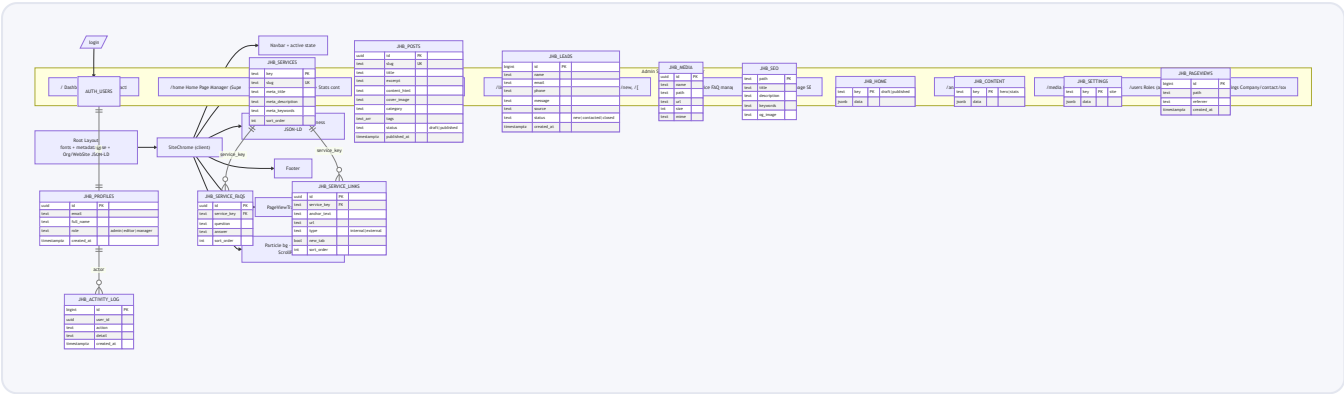
2. Public Website Architecture

Routes (implemented)

<code>/</code>	Home (Hero, Partners, About, Services, Stats, Testimonials, Process, Founder, Client Logos, FAQ, Blog preview, CTA, Contact)
<code>/about</code>	About page
<code>/contact</code>	Contact page (form → <code>jhb_leads</code>)
<code>/services</code>	Services index (grid)
<code>/services/[slug]</code>	Dynamic service page (SSR, <code>dynamicParams</code>)
<code>/blog</code>	Blog listing + <code>?q=</code> search
<code>/blog/[slug]</code>	Blog article (<code>BlogPosting</code> schema, related, share)
<code>/sitemap.xml</code>	Dynamic sitemap
<code>/robots.txt</code>	Robots

📡 **Planned:** `/privacy`, `/terms`, dedicated landing pages, blog **category** pages (category is stored on posts; no per-category route yet).

Page composition



Reusable components (apps/website/src/components)

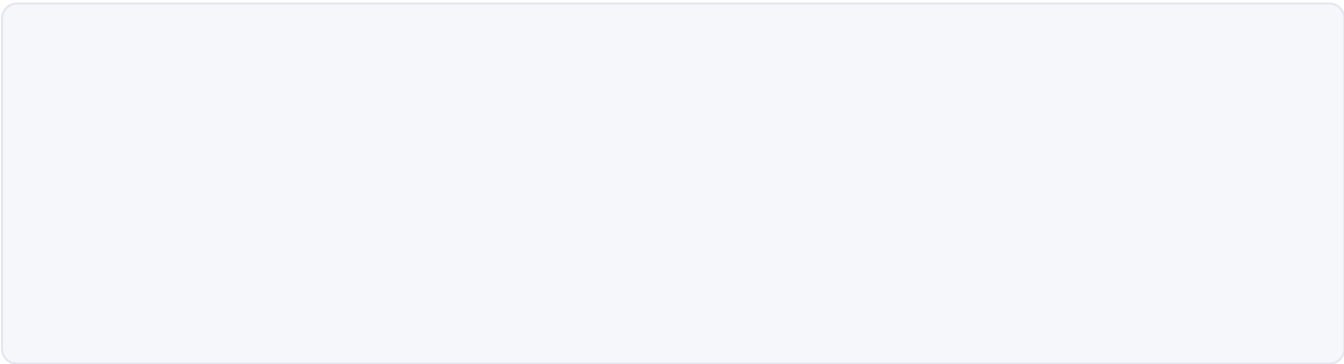
Navbar , Footer , SiteChrome , Hero , Partners , AboutSection , Services , Stats , Testimonials , Process , FounderPerspective , ClientLogos , Faq , FaqAccordion , BlogPreview , PostCard , ShareButtons , CtaSection , Contact , OfficeLocation , Breadcrumbs , ServiceDetail , Reveal , SectionHeading , Icon , ParticleBackground , CursorGlow , ScrollProgress , PageViewTracker .

Rendering & data flow

- Pages are **server components**; they call `@jhb/shared/*-server` functions which use the cookie-aware Supabase client → effectively **SSR/dynamic** so admin edits appear on next request.
- Home/Hero/Stats/Settings/Services/FAQ/Links are read from DB with **code-based fallbacks** (`data.ts`) so the site renders even before any admin edit.

3. Super Admin Dashboard Architecture

Modules (implemented routes in apps/admin)



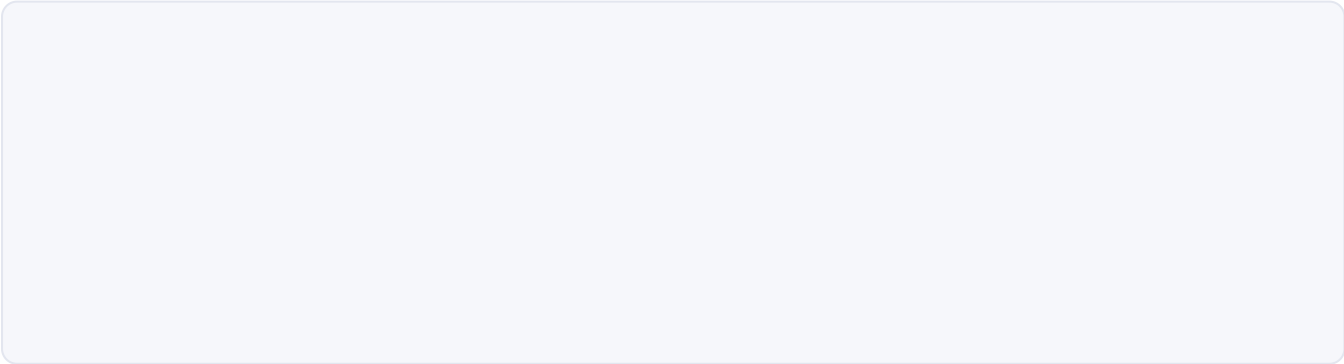
Module	Backing table(s)	Capabilities
Dashboard	jhb_leads , jhb_pageviews , jhb_media , jhb_activity_log	KPI counts + recent activity
Home Page Mgmt	jhb_home (draft/published)	hero, about, services heading, service cards, CTA + live preview , publish
Content	jhb_content (hero , stats)	hero text + animated stats
Services Mgmt	jhb_services	slug, meta title/desc/keywords (content base in data.ts)
Link Mgmt	jhb_service_links	anchor→URL, internal/external, validation, auto-link on page
Blog Mgmt	jhb_posts	CRUD, draft/publish, cover image, category, tags, SEO
FAQ Mgmt	jhb_service_faqs	per-service add/edit/delete + drag reorder, FAQ schema
SEO Mgmt	jhb_seo	per-path title/desc/keywords/OG; sitemap & robots auto
Lead Mgmt	jhb_leads	inbox, status (new/contacted/closed), delete
Media Library	jhb_media + bucket	drag-drop upload, compression, copy URL, delete
User Mgmt	jhb_profiles	view users, change role (admin only)
Settings	jhb_settings	company, contact, hours, socials

 **Planned (from spec, not yet built):** standalone Service Categories, Case Studies module, Quote-request lead type, Redirects manager, video/document media types.

4. Database Architecture

Engine: PostgreSQL (Supabase). All tables prefixed `jhb_` (the project is shared with another app). **RLS enabled on every table.**

ER diagram (as-built)



Mapping to the requested conceptual schema

Requested entity	As-built table	Notes
users + roles	<code>jhb_profiles</code> (role column) + <code>auth.users</code>	role enum, not a separate roles table
services	<code>jhb_services</code> (+ rich content in <code>data.ts</code>)	DB stores slug/SEO/order
service_categories	 Planned	not yet a table
service_faqs	<code>jhb_service_faqs</code>	✓
blogs	<code>jhb_posts</code>	✓
blog_categories / blog_tags	<code>category</code> text + <code>tags</code> text[] on <code>jhb_posts</code>	denormalised; tables 
testimonials / clients / case_studies	<code>jhb_home.data</code> (testimonials) + <code>data.ts</code>	dedicated tables 
leads / contact_submissions	<code>jhb_leads</code> (<code>source</code> distinguishes)	✓
media_library	<code>jhb_media</code> + Storage bucket	✓
seo_settings	<code>jhb_seo</code> + per-service meta in <code>jhb_services</code>	✓
redirects	 Planned	
settings	<code>jhb_settings</code>	✓

Authorization helper: `public.jhb_role()` (SECURITY DEFINER) returns the caller's role for RLS policies;
`jhb_handle_new_user()` trigger seeds a profile on signup.

5. API Architecture

The app uses **Next.js Server Actions** for writes and the **Supabase server client** for reads. Supabase also exposes an auto-generated **PostgREST** API (`/rest/v1/<table>`) protected by the same RLS. The conventional REST contract below maps to these.

Server Actions (admin) — `apps/admin/src/app/actions.ts`

Action	Conceptual REST	Auth
<code>saveContent(key,data)</code>	<code>PUT /api/content/:key</code>	staff
<code>saveSettings(data)</code>	<code>PUT /api/settings</code>	staff
<code>saveHomeDraft</code> / <code>publishHome</code>	<code>PUT /api/home/draft</code> · <code>POST /api/home/publish</code>	admin
<code>saveService(key,...)</code>	<code>PUT /api/services/:key</code>	staff
<code>savePost</code> / <code>deletePost</code>	<code>POST/PUT /api/blogs</code> · <code>DELETE /api/blogs/:id</code>	staff
<code>saveServiceFaqs(key,items)</code>	<code>PUT /api/services/:key/faqs</code>	staff
<code>saveServiceLinks(key,items)</code>	<code>PUT /api/services/:key/links</code>	staff
<code>saveSeo(path,...)</code>	<code>PUT /api/seo</code>	staff
<code>updateLeadStatus</code> / <code>deleteLead</code>	<code>PATCH/DELETE /api/leads/:id</code>	staff
<code>uploadMedia</code> / <code>deleteMedia</code>	<code>POST/DELETE /api/media</code>	staff
<code>updateUserRole(id,role)</code>	<code>PUT /api/users/:id/role</code>	admin

Public actions — `apps/website/src/app/actions.ts`

Action	Conceptual REST	Auth
<code>submitLead(input)</code>	<code>POST /api/leads</code>	public (RLS insert)
pageview insert (client)	<code>POST /api/pageviews</code>	public (RLS insert)

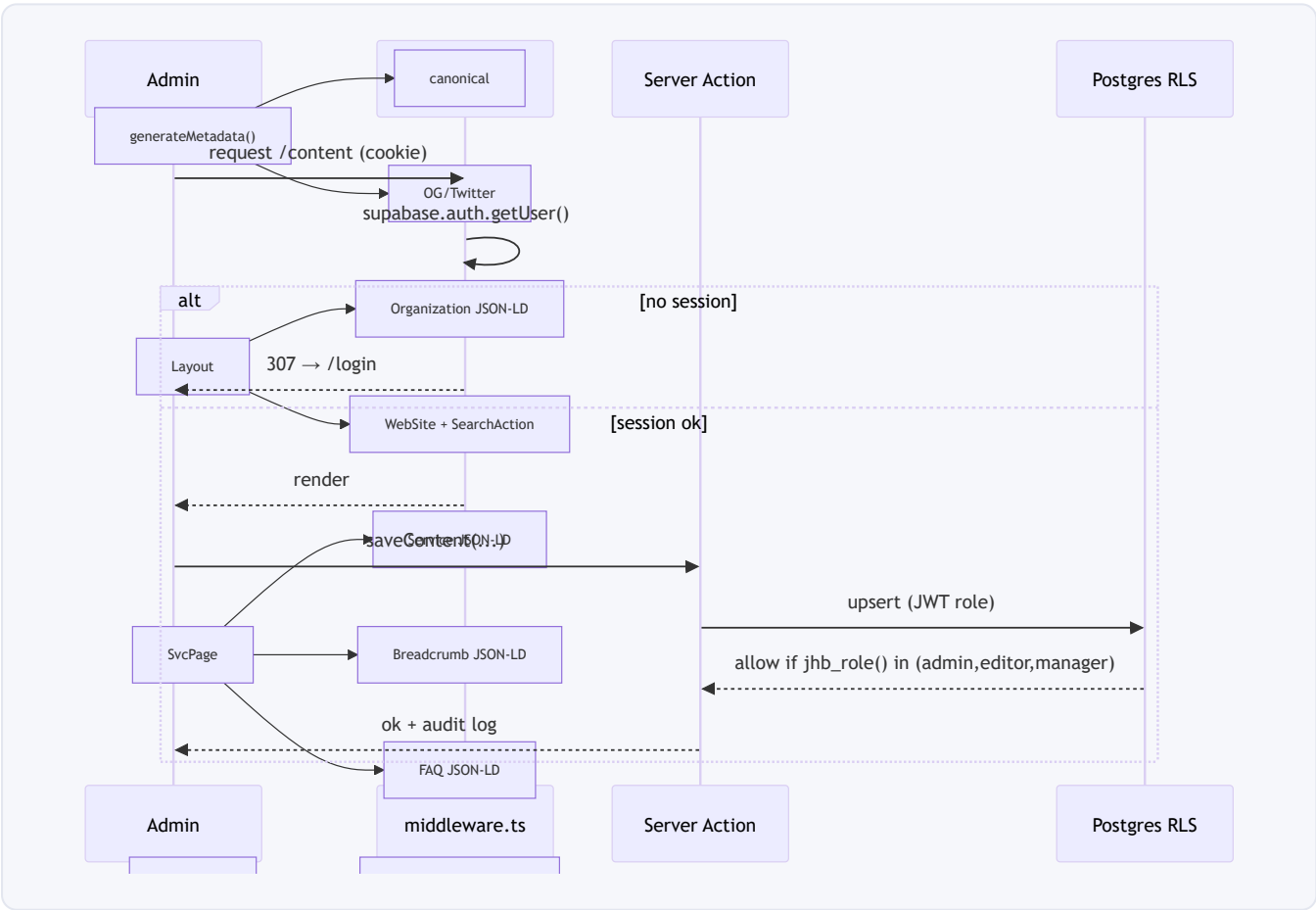
Reads (SSR) — `@jhb/shared/*-server`

`getHero/getStats/getSettings/getSeo/buildMetadata` , `getServices/getServiceBySlug` ,
`getPublishedPosts/getPostBySlug/getRelatedPosts` , `getServiceFaqs` , `getServiceAnchorLinks` , `getPublishedHome` .
 Equivalent to `GET /api/...` but executed server-side with RLS.

Permissions model: every endpoint is authorized by **RLS**, not app code — public can read published rows + insert leads/pageviews; `admin|editor|manager` can write content; `admin` only for home page + user roles.

6. SEO Architecture

Concern	Implementation
robots.txt	<code>app/robots.ts</code> — allow public, disallow <code>/admin /dashboard /login /api</code> , sitemap ref
XML sitemap	<code>app/sitemap.ts</code> — home, services (+slugs), blog (+published slugs), about, contact
Canonical URLs	<code>metadataBase</code> + <code>alternates.canonical</code> on every route
Open Graph / Twitter	site-wide defaults + per-page in <code>generateMetadata</code>
Organization + WebSite schema	root layout JSON-LD (+ <code>SearchAction</code> → <code>/blog?q=</code>)
LocalBusiness schema	<code>OfficeLocation</code> (NAP, hours, geo area, hasMap) on every page
Service schema	per service page (<code>Service</code> + provider)
BlogPosting schema	per blog article
FAQ schema	home FAQ + per-service <code>FAQPage</code>
Breadcrumb schema	<code>Breadcrumbs</code> component + per-page <code>BreadcrumbList</code>
Internal linking	Link Manager (<code>jhb_service_links</code>) auto-links anchor phrases in service content

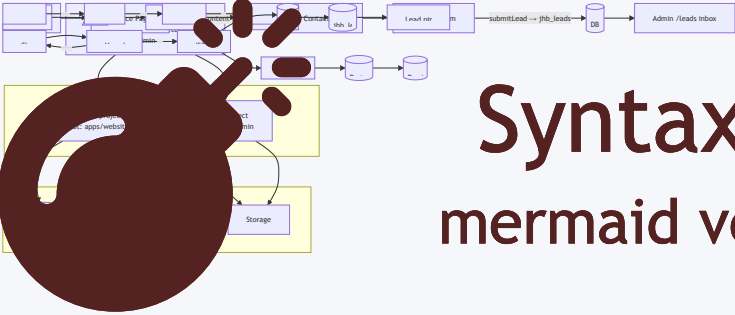


7. Security Architecture

Control	Status	Detail
Authentication	✓	Supabase Auth (GoTrue) issues JWT ; <code>@supabase/ssr</code> stores session in httpOnly cookies; <code>middleware.ts</code> refreshes + guards admin
Authorization (RBAC)	✓	RLS policies keyed on <code>jhb_role()</code> (admin/editor/manager); admin-only for Home + user roles
Admin isolation	✓	Separate app/origin; <code>robots</code> noindex; middleware redirects unauthenticated → <code>/login</code>
SQL injection	✓	All access via parameterised Supabase client / PostgREST (no string-built SQL)
XSS	⚠ Partial	React escapes by default; rich HTML (<code>content_html</code> , FAQ/CTA) is admin-authored and rendered via <code>dangerouslySetInnerHTML</code> — sanitize server-side (recommend <code>sanitize-html</code> /DOMPurify) 🔧
CSRF	✓ (mostly)	Server Actions use same-site POST + cookie; add explicit origin checks for any public POST 🔧
Rate limiting	🔧	Add on lead/pageview inserts (Vercel middleware / Supabase edge / WAF)
Audit logs	✓	<code>jhb_activity_log</code> records actor + action + detail on every admin mutation
Secrets	✓	Only the publishable/anon key in <code>NEXT_PUBLIC_*</code> ; service-role key never shipped

8. Deployment Architecture

As-built / recommended



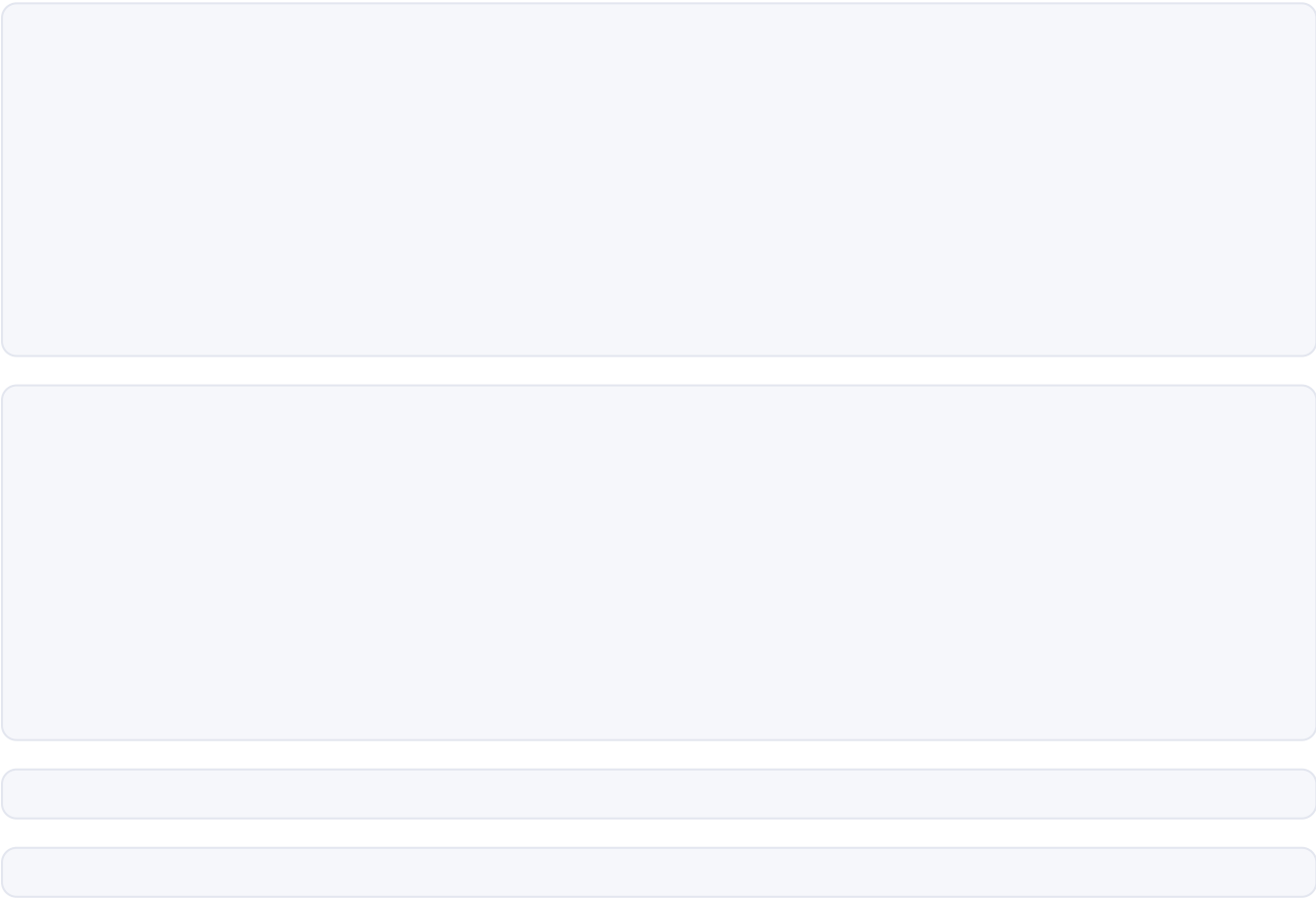
Syntax error in text

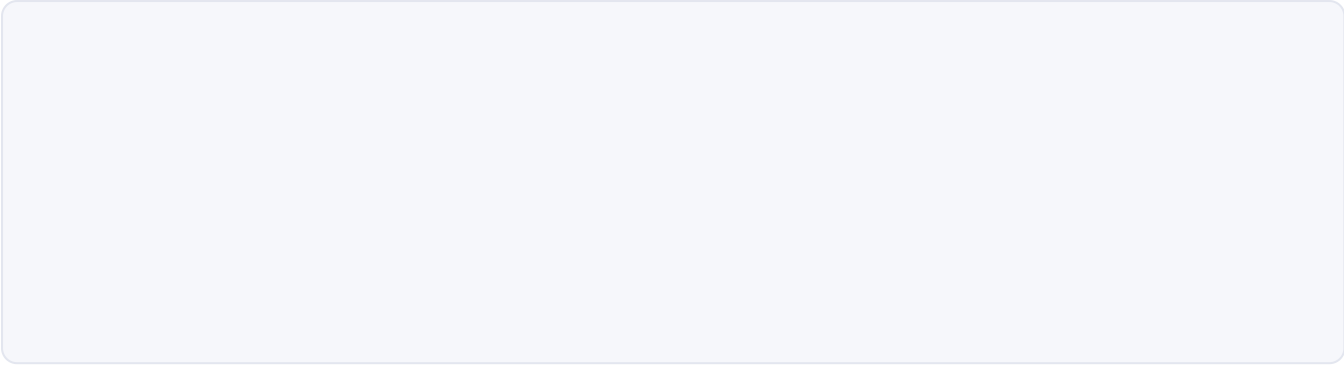
mermaid version 11.4.1

Concern	Recommended
Frontend	Next.js 15 + TypeScript + Tailwind (as built)
Backend	Supabase (Postgres + Auth + Storage + RLS) — replaces Express/Nest
Storage	Supabase Storage (Cloudinary/S3 optional for transforms)
Hosting	Vercel — two projects (website, admin), monorepo root dirs
Env	<code>NEXT_PUBLIC_SUPABASE_URL</code> , <code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code> , <code>NEXT_PUBLIC_SITE_URL</code> , <code>NEXT_PUBLIC_WEBSITE_URL</code>
Monitoring	GA4 + Google Search Console + Sentry (error tracking) 🛟
CI/CD	Vercel Git integration; preview deploys per PR

If a dedicated backend is later required (the spec's Node/Express/NestJS option): introduce a NestJS service in front of Postgres, move Server Actions → REST controllers, and keep RLS as defence-in-depth. Not needed at current scale.

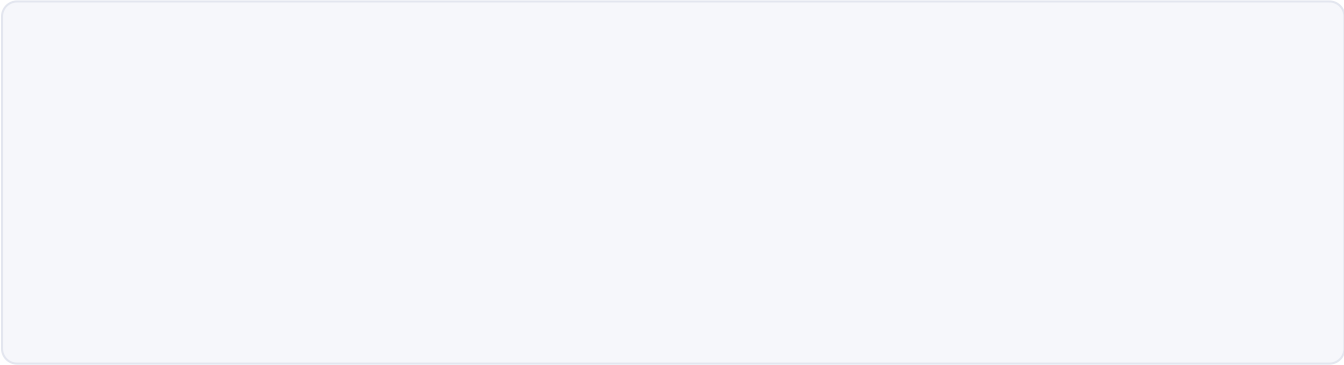
9. User Flow Diagrams





10. Scalability Roadmap

Stage	Traffic	Strategy
Launch	~100/day	Current setup: Vercel + Supabase free/pro, SSR. No changes needed.
Growth	~1,000/day	Switch hot public pages from per-request SSR to ISR (revalidate) + on-demand revalidation webhook from admin on publish; enable Vercel Edge cache.
Scale	~10,000/day	CDN for all static/ISR; add indexes on jhb_posts(status,published_at) , jhb_pageviews(created_at) ; move pageview writes to batched/edge; Supabase connection pooling (PgBouncer).
High scale	~100,000/day	Read replicas / Supabase compute upgrade; aggregate analytics into rollup tables (cron) instead of scanning jhb_pageviews ; rate-limit + WAF; object storage on CDN; consider splitting analytics to a dedicated pipeline.



Appendix A — Folder Structure

```
JHB-website/
├─ package.json                # npm workspaces root
├─ tsconfig.base.json
├─ docs/
│   └─ ARCHITECTURE.md        # this file
│   └─ migration-guide.md
├─ packages/shared/           # @jhb/shared (logic only)
│   └─ src/
│       ├── supabase/{client,server}.ts
│       ├── content.ts · content.server.ts
│       ├── data.ts            # canonical service content + nav
│       ├── services.server.ts
│       ├── home.ts · home.server.ts
│       ├── posts.ts · posts.server.ts
│       ├── faqs.ts · faqs.server.ts
│       ├── serviceLinks.ts · serviceLinks.server.ts
│       └─ types.ts
├─ apps/website/              # public (:3000)
│   └─ src/{app, components}
└─ apps/admin/                # admin (:3001)
    └─ src/{middleware.ts, app, components}
```

Appendix B — Best-Practice Recommendations (Production)

1. **Sanitize admin HTML** (`content_html` , FAQ/CTA) on save with DOMPurify/sanitize-html — closes the XSS gap.
2. **Set** `NEXT_PUBLIC_SITE_URL` to the real domain so canonicals/OG/sitemap use production URLs.
3. **Add** `logo.png` + `og.png` to `apps/website/public` (Organization logo + social share image).
4. **Switch public content pages to ISR** + on-demand revalidation webhook the admin calls after publish (decouples the two apps' caches).
5. **Add Sentry** to both apps; wire **GA4 + GSC** (first-party pageviews already exist).
6. **Rate-limit** public inserts (leads/pageviews) and add an origin check.
7. **Rotate** the seeded admin password; enable Supabase **leaked-password protection**.
8. **Pin & patch** Next.js (currently 15.1.6 — apply the security update on the next maintenance pass).
9. **Promote denormalised fields to tables** when needed: blog categories/tags, testimonials, clients/case-studies, redirects.
10. **Daily DB backups** (Supabase automatic on paid tiers) + periodic content JSON export.